

Phase 10 SOP – Namespaces, Resource Quotas & LimitRanges (EKS)

Overview

In the previous phases we learned:

- Pods
- Deployments
- Services
- Storage
- Rolling Updates
- Rollbacks
- HPA

Everything was running inside:

`default` namespace

This works for learning.

It does not work well in production.

Why Do We Need Phase 10?

Imagine a company has:

Development Team

Testing Team

Production Team

Monitoring Team

If everyone deploys into:

`default`

then:

- Resources mix together
- Teams interfere with each other
- No isolation exists
- One team can consume all cluster resources

Chaos.

What We Will Achieve

By the end of Phase 10 you will understand:

Namespace Isolation

Separate environments inside a single cluster.

dev

qa

prod

Resource Quotas

Limit how much an environment can consume.

Example:

Maximum 3 Pods

LimitRanges

Limit how much a single Pod can consume.

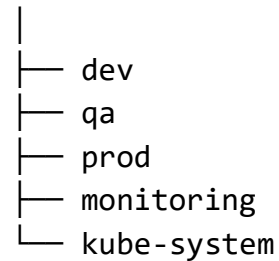
Example:

Max CPU = 500m

Max Memory = 512Mi

Real Production Architecture

EKS Cluster



All namespaces share:

Worker Nodes

Networking

Control Plane

but remain logically isolated.

Phase 10A – Understanding Namespaces

Objective

Learn what namespaces are and why Kubernetes uses them.

View Existing Namespaces

```
kubectl get ns
```

Output:

```
default
kube-system
kube-public
kube-node-lease
```

Understanding System Namespaces

default

Your application workloads.

Examples:

```
nginx-deployment
configmap-demo
ebs-demo
```

kube-system

Core Kubernetes components.

Examples:

```
coredns
kube-proxy
aws-node
metrics-server
ebs-csi-controller
```

kube-public

Public cluster information.

Rarely used by application teams.

kube-node-lease

Node heartbeat information.

Used internally by kubelet.

Create Development Namespace

```
kubectl create namespace dev
```

Verify:

```
kubectl get ns
```

Output:

```
default
dev
kube-system
...
```

Create Production Namespace

```
kubectl create namespace prod
```

Verify:

```
kubectl get ns
```

Output:

```
default
dev
prod
kube-system
...
```

Key Learning

Creating namespaces does NOT create:

New Cluster ❌

New Nodes ❌

New VPC ❌

It creates: Logical Isolation ✅

Phase 10B – Deploying Into Namespaces

Objective

Deploy identical applications into different namespaces.

Verify Dev Namespace Is Empty

```
kubectl get pods -n dev
```

Output:

No resources found

Create nginx-dev.yaml

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  name: nginx-deployment
```

```
  namespace: dev
```

```
spec:
```

```
  replicas: 1
```

```
  selector:
```

```
    matchLabels:
```

```
      app: nginx
```

```
  template:
```

```
    metadata:
```

```
      labels:
```

```
        app: nginx
```

```
    spec:
```

```
      containers:
```

```
        - name: nginx
```

```
          image: nginx:latest
```

Deploy

```
kubectl apply -f nginx-dev.yaml
```

Verify Deployment

```
kubectl get deployments -n dev
```

Expected:

```
nginx-deployment
```

Verify Pod

```
kubectl get pods -n dev
```

Expected:

```
nginx-deployment-xxxxx
```

Proof Of Namespace Isolation

Check all deployments:

```
kubectl get deployments -A
```

Output:

NAMESPACE	NAME
default	nginx-deployment
dev	nginx-deployment

Same name, Same cluster, No conflict.

Key Learning

Kubernetes identifies resources as:

Namespace + Resource Name

Example:

```
default/nginx-deployment
```

```
dev/nginx-deployment
```

These are different objects.

Phase 10C – Resource Quotas

Objective

Prevent an environment from consuming unlimited resources.

Production Problem

Developer accidentally deploys:

```
replicas: 500
```

Kubernetes tries creating:

```
500 Pods
```

Potential cluster outage.

Solution

ResourceQuota.

Create quota.yaml

```
apiVersion: v1
```

```
kind: ResourceQuota
```

```
metadata:
```

```
  name: dev-quota
```

```
  namespace: dev
```

```
spec:
```

```
  hard:
```

```
    pods: "3"
```

Apply

```
kubectl apply -f quota.yaml
```

Verify

```
kubectl get resourcequota -n dev
```

Detailed View

```
kubectl describe resourcequota dev-quota -n dev
```

Example:

Resource	Used	Hard
pods	1	3

Test Quota

Scale deployment:

```
kubectl scale deployment nginx-deployment `
  --replicas=3 `
  -n dev
```

Verify

```
kubectl get pods -n dev
```

Expected:

3 Pods Running

Try Exceeding Quota

```
kubectl scale deployment nginx-deployment `
  --replicas=4 `
  -n dev
```

Observe Deployment

```
kubectl get deployment nginx-deployment -n dev
```

Output:

3/4

Meaning:

Desired = 4

Actual = 3

Check ReplicaSet Events

```
kubectl describe rs <replicaset-name> -n dev
```

Example:

FailedCreate

exceeded quota: dev-quota

requested: pods=1

used: pods=3

limited: pods=3

Key Learning

ResourceQuota controls:

Namespace Consumption

Example:

Maximum Pods

Maximum CPU

Maximum Memory

Maximum PVCs

Phase 10D – LimitRanges

Objective

Prevent a single Pod from consuming excessive resources.

Production Problem

A namespace may be limited to:

20 CPUs

But one Pod requests:

10 CPUs

and monopolizes resources.

Solution

LimitRange.

Create limitrange.yaml

```
apiVersion: v1
kind: LimitRange

metadata:
  name: dev-limit-range
  namespace: dev

spec:
  limits:
    - type: Container

    default:
      cpu: "500m"
      memory: "512Mi"

    defaultRequest:
      cpu: "100m"
      memory: "128Mi"
```

Apply

```
kubectl apply -f limitrange.yaml
```

Verify

```
kubectl describe limitrange dev-limit-range -n dev
```

Output:

Container	cpu	100m	500m
Container	memory	128Mi	512Mi

Meaning

If a developer forgets:

resources: Kubernetes automatically injects:

```
resources:
  requests:
    cpu: 100m
    memory: 128Mi
```

```
limits:
  cpu: 500m
  memory: 512Mi
```

Proving LimitRange Works

Scale deployment back to:

```
kubectl scale deployment nginx-deployment `
  --replicas=2 `
  -n dev
```

to free quota space.

Create nginx-test.yaml

```
apiVersion: v1
kind: Pod
```

```
metadata:
  name: nginx-test
  namespace: dev
```

```
spec:
  containers:
  - name: nginx
    image: nginx
```

Notice:

No resources section

Deploy

```
kubectl apply -f nginx-test.yaml
```

Inspect Pod

```
kubectl get pod nginx-test -n dev -o yaml
```

Observe:

```
resources:
  limits:
    cpu: 500m
    memory: 512Mi

  requests:
    cpu: 100m
    memory: 128Mi
```

Automatically added by Kubernetes.

Key Learning

LimitRange controls:

Per Pod

Per Container

resource consumption.

Phase 10E – Storage & Namespace Behavior

Objective

Understand how PVCs, PVs and namespaces work together.

Important Concepts

PV

Cluster Scoped.

```
kubectl get pv
```

No namespace.

PVC

Namespace Scoped.

Examples:

```
default/ebs-pvc
```

```
dev/ebs-pvc
```

prod/ebs-pvc

Different objects.

Dynamic Provisioning Flow

When a PVC is created:

PVC



StorageClass



PV



AWS EBS Volume

Example

Default Namespace

default/ebs-pvc

Creates:

PV #1



EBS Volume #1

Dev Namespace

dev/ebs-pvc

Creates:

PV #2



EBS Volume #2

Prod Namespace

prod/ebs-pvc

Creates:

PV #3

↓

EBS Volume #3

Who Creates The Volume?

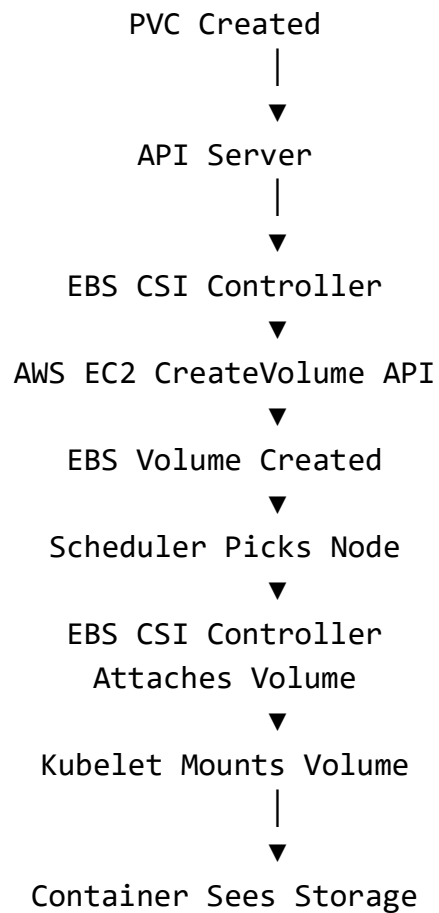
Not Kubernetes directly.

The:

EBS CSI Controller

does.

Storage Provisioning Flow



Who Gives Permission?

IRSA.

EBS CSI Controller



Service Account



IAM Role



AWS APIs

Required permissions include:

ec2:CreateVolume

ec2:AttachVolume

ec2:DetachVolume

ec2:DeleteVolume

ResourceQuota vs LimitRange

Feature	Purpose
Namespace	Logical isolation
ResourceQuota	Controls total namespace consumption
LimitRange	Controls individual pod/container consumption
PVC	Requests storage
PV	Actual storage resource
StorageClass	Defines provisioning behavior
EBS CSI Controller	Creates and attaches AWS volumes